

CreditSetu: A Dual-Path Credit Scoring & Lending Marketplace for Microfinance

Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Maanya Garg, CSED* Kashvi Bansal, CSED[†] Shreesh Gupta, CSED[‡]
Shivam Raj, CSED[§]

September 16, 2026

1 Higher-order goal

... secondary framing

This project aims to widen access to formal credit for underserved borrowers in India – particularly members of rural Self-Help Groups (SHGs) and independent borrowers who have no prior formal credit history. The immediate engineering objective is to translate *trustworthy, explainable credit assessment* into a measurable, deployable software system that a microfinance lender could plausibly adopt.

2 Time-to-value

... primary heuristic

- We deliver meaningful increments quickly: a working synthetic-data pipeline, a trained scoring model, and a queryable API were built and exercised end-to-end before any UI existed.
- We prioritize fast feedback over perfection by validating each pipeline stage against real (synthetic) data end-to-end – training on real data, scoring real individuals, hitting the live API, and screenshotting the live frontend – rather than reading code and assuming correctness.
- Success is defined by what can be delivered and verified working in each iteration, then improved: the scoring engine and open marketplace shipped first; role-separated logins and authorization are the current iteration.

3 Problem Statement

Microfinance lending in India serves two kinds of borrowers very unevenly. Common pain points include:

*Roll No: 1024030564, Email: <TODO: confirm official Thapar email>

[†]Roll No: 1024030563, Email: <TODO: confirm official Thapar email>

[‡]Roll No: 1024030565, Email: <TODO: confirm official Thapar email>

[§]Roll No: 1024031140, Email: <TODO: confirm official Thapar email>

- **No credit history for new/independent borrowers:** a borrower with no SHG affiliation and no prior loans has nothing a lender can score against – the classic cold-start problem.
- **Opaque scoring:** where informal scoring does exist (e.g. group-level trust), it rarely explains *why* a given score is what it is, which undermines both borrower trust and lender diligence.
- **No structured SHG-lender relationship:** lenders either lend to anyone in a group or nobody – there is no formal, auditable approve/reject linkage between a specific SHG and a specific lender.
- **Everyone sees everyone’s data:** in an open system, a borrower, a lender, and an SHG all see the same undifferentiated data – there is no role-appropriate, access-controlled view for each stakeholder.

Why this matters now (contextual relevance): SHG-based microfinance already operates at large scale in India; a system that lets a borrower’s *own* repayment history build their score, while still giving SHG members a fair trust-based head start, directly targets an active fairness and access gap in that ecosystem.

4 Proposed Solution

... *Software Proposition*

4.1 Overview

Build a **web-based** dual-path credit scoring and lending-marketplace system, CreditSetu, with the following components:

- **Dual-path scoring engine:** an SHG-linked borrower’s score bootstraps from their group’s track record; an independent borrower starts from a flat, lower base and builds purely from their own repayment and savings behaviour.
- **Explainable scoring:** an XGBoost classifier with SHAP explanations, so every score comes with a feature-level breakdown of why it is what it is, not just a number.
- **SHG-lender linkage:** lenders formally request, and SHGs approve or reject, a lending relationship – not open lending to anyone.
- **Loan marketplace:** lenders can make offers to eligible borrowers, and borrowers can directly request loans that are broadcast to every lender they are currently eligible for.
- **Role-separated access:** four distinct, login-gated views – Borrower, Lender, SHG, and Admin – each seeing only the data appropriate to that role, enforced server-side rather than just hidden in the UI.

4.2 Core workflow

1. A borrower’s repayment, savings, and (if applicable) SHG-attendance history feeds a feature pipeline.
2. The scoring engine blends a cold-start prior (SHG or independent base) with the trained model’s output as repayment history accumulates.
3. Lenders discover SHGs and borrowers they are formally linked to (or independents, who are open to all lenders by design), and either make offers or respond to borrower-initiated requests.

4. Every role logs into its own dashboard: a borrower sees only their own score/history/suggestions and their own SHG; a lender sees SHGs by area and its own linked members and requests; an SHG sees its own members and linked lenders; an admin sees read-only platform-wide oversight.

4.3 Operational constraints for an educational setting

- Use synthetic but behaviourally realistic data (no real borrower PII), generated and versioned as part of the pipeline, not hand-curated.
- Avoid dependence on novel research algorithms; XGBoost + SHAP are mature, well-understood tools – the engineering focus is the scoring pipeline, access control, and marketplace correctness.
- Design for deployability on common web hosting (SQLite for the demo, a one-line DATABASE.URL swap to Postgres for concurrent production use).

5 Solution Approach

... Engineering focus

This is an engineering course project, so we focus on a correct, explainable, access-controlled pipeline rather than algorithmic novelty.

5.1 Scoring pipeline

... non-research

- Feature engineering turns each individual's loans, repayments, savings, and SHG attendance into features such as repayment rate, savings regularity, and peer repayment rate.
- An XGBoost classifier is trained on SHG-linked individuals with sufficient repayment history, predicting default risk; SHAP's `TreeExplainer` attributes every prediction to its contributing features.
- A cold-start blend interpolates between a pure prior (SHG bootstrap or independent base) and the trained model's output as an individual accumulates repayment events, so a score is never asked to extrapolate from data it doesn't have.

5.2 Web architecture

... web-based solution

A 3-tier web architecture:

- **Frontend:** a React (Vite) single-page app with four role-gated dashboards and a public landing page.
- **Backend API:** FastAPI, with bearer-token session auth per role and server-side authorization scoping every query to the authenticated caller's own identity.
- **Database:** SQLAlchemy over SQLite (demo) / Postgres (production), with an append-only score history table so a borrower's score-over-time and its SHAP explanation are always reconstructable.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and lint on every change.

- Produce a repeatable, from-scratch pipeline run (data generation → training → batch scoring → serving) as the release artifact, not just a code diff.
- Enable safe, frequent releases of incremental features (the role-separated login redesign is itself one such increment, layered on a previously-working open system).

6 Evaluation Criterion

...measurable and attributable

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Access-control correctness: the fraction of cross-role/cross-identity data-access attempts that are correctly rejected (401/403) rather than served.

- **Target:** 100% of out-of-scope requests (e.g. a borrower fetching another borrower’s data, or a lender fetching an unlinked SHG’s member list) are rejected.
- **Attribution:** verified directly against the live API with authenticated and unauthenticated requests per role.

6.2 Secondary evaluation metrics

- **Explanation coverage:** percentage of delivered scores that have a non-empty, correctly normalised SHAP explanation (points sum to `score - SCORE_MIN`).
- **Cold-start behaviour:** score assigned to a borrower with zero repayment events is exactly the configured prior (SHG bootstrap or independent base), with no model call.
- **Marketplace liveness:** lender accept/reject decisions on a loan request are reflected in both the lender’s and the borrower’s view without a stale read.
- **Lender reputation soundness:** a lender’s computed reputation score responds monotonically to a lower offered interest rate relative to the platform average, given the same activity level.

6.3 Criteria added after the payment-gateway and lender-targeting iteration

...problem / solution / evaluation criterion

Problem: Borrowers had no way to build real credit history through the app – repayment history only ever came from offline synthetic data generation, never from a live borrower action.

Solution: A live repayment flow (wallet balance + dummy payment gateway) that pays down a real EMI.

Evaluation criterion: A successful payment changes the borrower’s credit score within the same request that processes it – verified on a real account: score moved from 596 (Medium risk) to 886 (Very Low risk) once enough on-time repayment history accumulated. A wallet balance insufficient to cover the borrower’s *total* dues this cycle blocks payment of every active loan, not just the one clicked.

Problem: A loan request always broadcast to every eligible lender, with no way to direct it at one preferred lender.

Solution: Optional lender targeting on a request, with a retarget path if declined.

Evaluation criterion: A targeted request is visible only in the targeted lender’s queue (verified: other eligible lenders do not see it), and a declined targeted request can be retargeted rather than becoming a dead end.

Problem: GET /api/shgs (the SHG browse list used by both the lender and admin dashboards) computed each SHG’s aggregate repayment/attendance rate as a Python property that walked `members` -> `loans` -> `repayment_events` per SHG – an N+1 access pattern that, for 51 SHGs, materialised on the order of 25,000+ `RepaymentEvent` ORM objects on every request.

Solution: Replaced the per-SHG Python walk with three GROUP BY SQL aggregate queries computed once for all SHGs, joined back to each row by id.

Evaluation criterion (measured, not estimated): median response time for GET /api/shgs against the live dataset (51 SHGs, 1,079 individuals) dropped from 1,101–1,322 ms to 14.5–26.7 ms across repeated runs – roughly a 75× improvement – with byte-for-byte identical response content verified before and after.

6.4 Pilot validation plan

... *high-level*

- Exercise the full pipeline against realistic synthetic data (over a thousand individuals, dozens of SHGs, multiple lenders across several districts).
- Validate each role’s dashboard end-to-end in a live browser session (not just unit tests) – screenshotting and checking console/network errors for each of the four logins.

7 Scalability

... *with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the system supports growth in individuals, SHGs, and lenders by:
 - decoupling frontend and backend behind a REST API,
 - indexing every foreign key (a gap we found and fixed during the build),
 - paginating and server-side filtering list endpoints instead of client-side filtering over one page.
2. **Operationally deployable (practically):** the system runs on common web hosting:
 - a one-line environment-variable swap from SQLite to a managed Postgres instance,
 - stateless bearer-token sessions rather than server-side sticky state,
 - CI/CD pipeline for repeatable build/deploy.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this project:

- FastAPI for REST APIs, with automatic OpenAPI docs.
- SQLAlchemy ORM over SQLite/Postgres.
- XGBoost and SHAP for the scoring model and its explanations.
- scikit-learn (k-means, isolation forest) for geo-clustering and anomaly detection.
- React + Vite for the frontend.
- A token-based session-auth pattern (no new dependency needed for a demo-scale system).

We use these mature components to focus on pipeline correctness, explainability, and access control rather than inventing new infrastructure.

9 Project scope and deliverables

... *iteration-friendly*

9.1 Initial deliverable (shipped)

... *first iterations*

- Synthetic data generation (individuals, SHGs, lenders, loans, repayments, savings).
- Trained XGBoost + SHAP scoring engine, with cold-start bootstrap blending for both borrower paths.
- SHG-lender linkage, lender matching, and a loan offer/request marketplace.
- k-means district clustering and isolation-forest anomaly detection.
- A first working frontend across all of the above.

9.2 Current iteration (prototype stage)

- Four separate, role-gated logins (Borrower, Lender, SHG, Admin) replacing the previously-open single interface.
- Server-side authorization enforced on every endpoint, scoped to the authenticated caller's own identity.
- A lender reputation/incentive score rewarding competitive interest rates, informed by real-world analogues (RBI priority-sector-lending and co-lending incentive structures).
- A redesigned landing page surfacing real, data-derived borrower-improvement stories, SHG/lender spotlights, and district-level score trends.

9.3 Subsequent deliverables

- Periodic re-scoring so genuine score-history trends accumulate over time (currently each individual has a small number of scoring snapshots, since the pipeline has only been run a handful of times).
- Admin account-management/moderation actions beyond the current read-only oversight view.
- Real password hashing/short-lived signed tokens in place of the deliberately-minimal demo bearer-token scheme.

10 Risks and mitigations

- **Cold-start data scarcity:** mitigated by an explicit bootstrap-to-model blend rather than forcing the model to score borrowers it has no signal on.
- **Fairness of the independent-vs-SHG base score gap:** this is a stated policy parameter (INDEPENDENT_BASE_SCORE vs. the SHG bootstrap formula), not something the data derived – flagged explicitly rather than hidden, and a candidate for a sensitivity analysis.
- **Demo-data limitations for time-series features:** the synthetic dataset is scored in discrete batch runs, so "score improved over time" features are only as rich as the number of real scoring passes that have actually been run – we surface real deltas only, never fabricated ones, even when that means a feature section is temporarily sparse.
- **Security of role-based auth:** mitigated by enforcing authorization server-side (401/403) on every endpoint, not relying on the frontend to hide UI elements.

11 Summary

This project proposes a deployable, explainable, dual-path credit-scoring and lending-marketplace platform for microfinance in India. It meets the course criteria by delivering value iteratively and verifying each stage end-to-end against real data, using CI/CD to support frequent safe releases, and defining measurable evaluation criteria (access-control correctness, explanation coverage, cold-start correctness, and marketplace liveness). It remains focused on engineering pipeline correctness, explainability, and access control rather than algorithmic novelty.